TRACKORA

Frontend Development Plan

Goal: build the mobile and admin interfaces first while the backend developer works in parallel. Use realistic mock data for UI development, but isolate it behind services so real API integration does not require rewriting screens.

Area	Technology	Use
Mobile app	Flutter + Dart	Android/iOS salesperson app, forms, GPS permission flow and offline UX
Admin web	Next.js + TypeScript	Responsive dashboards, team management, approvals and reports
Web styling	Tailwind CSS + reusable components	Consistent layout, forms, tables, dialogs and status chips
Local mobile data	SQLite / Drift	Drafts and queued submissions when offline
Integration layer	Typed API service/repository	Swap mock implementation for real API calls

Core rule: mock data is temporary and must be clearly identified. Mock login, fake GPS, local-only approval and sample analytics are not production functionality.

1. Mobile app scope - Trackora Field

• Authentication: sign-in, validation, session loading, expired-session and sign-out states.

• Dashboard: attendance state, visits submitted/approved/pending, target progress and quick actions.

• Attendance: check-in/out UI, permission explanations, location loading/errors and timestamps.

• New visit: customer search/create, multiple contacts, visit type, notes, optional deal value, photo/audio attachment and GPS evidence.

• Visit history: date/status filters, visit details, rejection reason and permitted edit/resubmit flow.

• Performance: target progress, leaderboard, ranking period and achievements.

• Profile/settings: account information, notification preferences and app settings.

• Offline UX: draft, queued, syncing, synced and failed states with safe retry/conflict messages.

2. Admin web scope - Trackora Control Center

• Overview: KPI cards, date/team filters, pending approvals and activity summaries.

• Employees and teams: searchable/paginated lists, employee details, role and team assignment screens.

• Visit approval centre: review queue, visit detail, approve/reject confirmation and required rejection reason.

• Historical tracking: select any date and inspect attendance, visits, contacts, notes, timestamps, location evidence and decision history.

• Targets and rankings: target configuration forms, progress and transparent ranking period/rules.

• Reports: filters, loading/empty/error states and export actions connected to backend when available.

• Settings: company timezone and notification policy screens when supported by the backend.

3. Suggested frontend structure

```
trackora/
■■■ apps/
■   ■■■ mobile/                # Flutter
■   ■   ■■■ lib/
■   ■       ■■■ core/           # routing, theme, networking, errors
■   ■       ■■■ shared/         # reusable widgets and models
■   ■       ■■■ features/
■   ■           ■■■ auth/ dashboard/ attendance/
■   ■           ■■■ visits/ customers/ performance/ profile/
■   ■■■ admin-web/             # Next.js
■       ■■■ app/               # routes and layouts
■       ■■■ components/        # shared UI
■       ■■■ features/          # feature modules
■       ■■■ lib/api/           # API client and error handling
■       ■■■ lib/mock-data/     # temporary mock repositories
■       ■■■ types/             # TypeScript contracts
■■■ docs/api-contracts/        # shared API definitions
```

Screens/components should not call HTTP directly. Route calls through a service/repository layer so mock and real implementations can be switched safely.

4. Frontend-first workflow

Phase 1 - Design foundation

• Agree brand, typography, spacing, colors and navigation.

• Build reusable buttons, fields, cards, tables, status labels, dialogs and loading/empty/error states.

Phase 2 - Screen skeletons

• Implement routing and main screens with mock data.

• Check phone sizes and responsive admin layouts.

Phase 3 - Local interactions

• Implement form validation, search, sorting, filters, date selection, pagination and confirmation dialogs.

• Test success, validation, empty, loading, error and permission-denied states.

Phase 4 - API contract

• Agree endpoint paths, JSON fields, enums, date formats, pagination, auth and error responses with backend developer.

• Keep mock repositories interchangeable with real API repositories.

Phase 5 - Integration

• Connect authentication first, then attendance, visits, approvals, targets and reports.

• Handle timeout, unauthorized, validation and server errors.

5. API contract checklist

Topic	Agree before integration
Authentication	Session/token, expiry, refresh and logout behaviour
Visit fields	Required fields, customer/contact IDs, visit type, GPS, timestamps and attachment metadata
Statuses	Exact values and permitted transitions, e.g. DRAFT, PENDING, APPROVED, REJECTED
Errors	Consistent response shape, field validation and HTTP status codes
Lists	Pagination, sorting, filters, date/timezone semantics and empty results
Duplicate prevention	Idempotency key or equivalent safe-retry approach
Permissions	Which roles can view/change each resource; backend enforces independently

6. Frontend quality checklist

• No production credentials or secrets inside client code.

• Mock data and mock authentication are clearly labelled and disabled for production builds.

• Forms include validation; screens handle loading, empty, error and retry states.

• Large lists use pagination/server-side filtering.

• Mobile handles denied permissions and poor connectivity gracefully.

• Admin route protection improves UX but is not a replacement for backend authorization.

• Add tests for important validation, navigation and status display.

Handoff deliverables

Give the backend developer the API contract, enum values, sample request/response JSON, screen-specific data needs and validation rules. Keep a shared change log for API updates.

Frontend-ready means: screens and local interactions work, mock data is isolated, API types are documented, and features can switch to real API services one by one.